06

JavaScript assíncrono e APIs

Apostila do aluno · duração da aula: 2h30

O que você vai aprender

- 1. Síncrono x assíncrono
- 2. Timers: setTimeout e setInterval
- 3. Callbacks e o callback hell
- 4. Promises
- 5. async e await
- 6. fetch: buscando dados de uma API
- 7. APIs de verdade: ViaCEP e usuários

Antes de começar

Esta aula usa funções e callbacks (Aula 4), objetos, arrays e JSON (Aula 1). No site **site-alunos-nu.vercel.app** o playground desta aula espera os timers e as Promises terminarem, e os exemplos e os últimos exercícios com `fetch` buscam dados de APIs de verdade (precisa de internet).

Prof. Walter Caciano · Greenn School

1 Síncrono x assíncrono

Síncrono: uma linha de cada vez, cada uma espera a anterior terminar. **Assíncrono:** você pede algo que demora (um timer, um dado da internet) e o programa **segue em frente**; quando ficar pronto, ele avisa.

Analogia: você pede uma pizza e vai arrumar a mesa. Não fica parado olhando para a porta.

```
console.log("1. Pedi a pizza");

setTimeout(() => {
  console.log("3. A pizza chegou!");
}, 1000);

console.log("2. Fui arrumar a mesa");   // aparece ANTES da pizza
```

2 Timers: `setTimeout` e `setInterval`

Função	O que faz
`setTimeout(fn, ms)`	roda **uma vez**, depois de `ms` milissegundos (1000 ms = 1 s)
`setInterval(fn, ms)`	roda **a cada** `ms`, para sempre
`clearTimeout(id)` / `clearInterval(id)`	cancela / para (o id é o que o set devolveu)

```
let contador = 3;
const id = setInterval(() => {
  console.log(contador);
  contador--;
  if (contador === 0) {
    clearInterval(id);        // sem isso, repete para sempre!
    console.log("Fogo!");
  }
}, 1000);
```

3 Callbacks e o callback hell

A forma antiga de "avisar quando terminar" era passar uma função de **callback**. Quando uma tarefa depende da outra, os callbacks ficam um dentro do outro, formando uma pirâmide difícil de ler: o **callback hell**.

```
buscarUsuario(1, (usuario) => {
  buscarPedidos(usuario, (pedidos) => {
    buscarEntrega(pedidos[0], (entrega) => {
      console.log(entrega);       // cada etapa, um nível mais fundo...
    });
  });
});
```

4 Promises

Uma **Promise** (promessa) representa um valor que **ainda vai chegar**, como o comprovante do pedido: ainda não é a pizza, mas é a promessa de que ela vem. Ela tem 3 estados:

Estado	Significado
pendente	ainda esperando
resolvida	deu certo: chamou `resolve(valor)`
rejeitada	deu erro: chamou `reject(erro)`

```
function buscarAluno(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id === 1) resolve({ nome: "Ana", nota: 9 });
      else reject(new Error("Aluno não encontrado"));
    }, 500);
  });
}

buscarAluno(1)
  .then((aluno) => console.log(aluno.nome))     // deu certo
  .catch((erro) => console.log(erro.message))   // deu erro
  .finally(() => console.log("Fim"));           // sempre
```

> **Encadeando:** se um `then` devolve um valor ou outra Promise, o próximo `then` recebe. As etapas ficam uma embaixo da outra, sem pirâmide, e um único `catch` no fim pega o erro de qualquer etapa.

5 `async` e `await`

Um jeito de escrever código com Promises que **parece síncrono**. `await` espera a Promise e já entrega o valor; só pode ser usado dentro de uma função `async`. Erros são tratados com `try/catch`:

```
async function mostrarAluno(id) {
  try {
    const aluno = await buscarAluno(id);    // espera aqui
    console.log(`${aluno.nome} tirou ${aluno.nota}`);
  } catch (erro) {
    console.log("Erro:", erro.message);
  } finally {
    console.log("Busca encerrada");
  }
}
```

Promise.all roda várias tarefas **ao mesmo tempo** e espera todas (com await em sequência, os tempos somam):

```
const [a, b] = await Promise.all([buscarAluno(1), buscarAluno(2)]);
```

6 `fetch` : buscando dados de uma API

`fetch(url)` busca dados na internet e devolve uma **Promise** com a resposta. A resposta chega como texto, então usamos `await resposta.json()` para virar objeto. Sempre confira `resposta.ok` (false em erros como 404).

```
async function buscarCep(cep) {
  try {
    const resposta = await fetch(`https://viacep.com.br/ws/${cep}/json/`);
    if (!resposta.ok) {
      console.log("Erro do servidor:", resposta.status);
      return;
    }
    const dados = await resposta.json();
    console.log(dados.logradouro, "-", dados.localidade);
  } catch (erro) {
    console.log("Falha na conexão:", erro.message);
  }
}

buscarCep("01001000");
```

> **Quiz do event loop:** o que aparece primeiro? Todo o código síncrono, depois os `then` de Promises já resolvidas, e por último os timers, mesmo com 0 ms. Teste no site o exemplo "Em que ordem aparece?".

7 APIs de verdade: ViaCEP e usuários

Uma **API** é um endereço que devolve **dados** em vez de página, quase sempre em **JSON**. A resposta do `fetch` tem `status` (200 ok, 404 não existe, 500 erro no servidor), `ok` (true para 2xx) e o corpo, lido com `.json()`. Duas APIs públicas e gratuitas que usamos na aula:

API	URL e o que devolve
ViaCEP (CEPs do Brasil)	`https://viacep.com.br/ws/01001000/json/` → `logradouro`, `bairro`, `localidade`, `uf`, `ddd`. CEP inexistente devolve `{"erro": "true"}` (status 200!).
BrasilAPI (dados do Brasil)	`brasilapi.com.br/api/feriados/v1/2026` (feriados), `/api/banks/v1/001` (banco pelo código; 404 se não existe), `/api/ddd/v1/81` (cidades do DDD), `/api/cep/v2/...`.
PokéAPI	`pokeapi.co/api/v2/pokemon/pikachu` → `name`, `height`, `weight`, `types[].type.name` (dados aninhados).
Open-Meteo (clima)	`api.open-meteo.com/v1/forecast?latitude=-8.05&longitude=-34.88&current_weather=true` → temperatura e vento **de agora** (parâmetros na URL).
AwesomeAPI (moedas)	`economia.awesomeapi.com.br/last/USD-BRL` → cotação em tempo real; os números vêm como texto (`Number()`).
GitHub	`api.github.com/users/octocat` → perfil público; 60 pedidos/hora sem login (403 se passar).
JSONPlaceholder (dados de teste)	`https://jsonplaceholder.typicode.com/users` → lista de 10 usuários com `name`, `email`, `address.city`, `company.name`.

```
async function buscarCep(cep) {
  const resposta = await fetch(`https://viacep.com.br/ws/${cep}/json/`);
  const dados = await resposta.json();
  if (dados.erro) {
    console.log("CEP não encontrado");
    return;
  }
  console.log(`${dados.logradouro}, ${dados.localidade} - ${dados.uf}`);
}
await buscarCep("01001000");   // Praça da Sé, São Paulo - SP
await buscarCep("00000000");   // CEP não encontrado

async function listarUsuarios() {
  const resposta = await fetch("https://jsonplaceholder.typicode.com/users");
  if (!resposta.ok) { console.log("Erro", resposta.status); return; }
  const usuarios = await resposta.json();
  usuarios.slice(0, 3).forEach((u) => console.log(`- ${u.name} (${u.address.city})`));
}
```

O padrão de sempre: fetch → conferir `ok` → `json()` → usar os dados, tudo dentro de `try/catch` (a conexão pode cair). É exatamente isso que o seu portfólio vai fazer no módulo de Laravel, só que consumindo a API que **você** vai construir.

7 Exercícios

Do 1 ao 3 você treina timers e callbacks; do 4 ao 7, Promises e async/await; do 8 ao 10 junta tudo com APIs "de mentira" (já prontas no código inicial); do 11 ao 16 você consome APIs de verdade: ViaCEP, usuários, feriados e bancos da BrasilAPI e a PokéAPI. Em cada um, compare a sua saída com a esperada — cada letra, espaço e acento conta.

01 setTimeout — TIMERS

Mostre "Início", agende "Depois de meio segundo" para 500 ms e mostre "Fim" logo abaixo.

Saída esperada: Início / Fim / Depois de meio segundo

02 setInterval — TIMERS

Com `setInterval` de 300 ms, mostre 3, 2, 1, pare com `clearInterval` e mostre "Fogo!".

Saída esperada: 3 / 2 / 1 / Fogo!

03 Callback — TIMERS

Complete `esperar(ms, callback)` com `setTimeout` e use para mostrar "Pronto!" depois de "Esperando...".

Saída esperada: Esperando... / Pronto!

04 Criando uma Promise — PROMISES

Crie `sortear(numero)`: resolve "Você ganhou!" se for maior que 5, senão rejeita com "Tente de novo". Teste 8 (then) e 2 (catch).

Saída esperada: Você ganhou! / Erro: Tente de novo

05 then e catch — PROMISES

Use a API `buscarAluno`: mostre o aluno 1 com then e o erro do aluno 3 com catch.

Saída esperada: Ana tirou 9 / Erro: Aluno não encontrado

06 async/await — PROMISES

Reescreva o exercício 5 com `async function`, `await` e `try/catch`.

Saída esperada: Ana tirou 9 / Erro: Aluno não encontrado

07 Função esperar — PROMISES

Crie `esperar(ms)` que devolve uma Promise e use `await esperar(500)` entre "Carregando..." e "Pronto!".

Saída esperada: Carregando... / Pronto!

08 Uma etapa depois da outra

JUNTANDO TUDO

Busque o aluno, depois as notas dele com `buscarNotas` e mostre a média (reduce).

Saída esperada: Média da Ana: 9.0

09 Promise.all

JUNTANDO TUDO

Busque o preço de caderno, caneta e mochila ao mesmo tempo com `Promise.all` e some.

Saída esperada: Total: R$ 115

10 Consumindo uma API

JUNTANDO TUDO

Com `fetchFalso` (igual ao fetch), confira `resposta.ok`, leia o JSON, filtre os aprovados e trate o erro 404.

Saída esperada: Aprovados: Ana, Bia / Erro 404

11 CEP de verdade (ViaCEP)

APIS REAIS

fetch no ViaCEP e mostre rua, bairro e cidade - UF.

Saída esperada: Rua: Praça da Sé / Bairro: Sé / Cidade: São Paulo - SP

12 CEP que não existe

APIS REAIS

`buscarCep(cep)` que trata o `dados.erro`.

Saída esperada: Praça da Sé, São Paulo / CEP não encontrado

13 Usuários de uma API

APIS REAIS

Liste a quantidade e os 3 primeiros usuários do JSONPlaceholder, conferindo `ok`.

Saída esperada: 10 usuários / - Leanne Graham (Gwenborough) ...

14 Feriados do ano (BrasilAPI)

APIS REAIS

Os 3 primeiros feriados de 2026 no formato DD/MM: nome (split na data).

Saída esperada: 01/01: Confraternização mundial / 16/02: Carnaval ...

15 Dois Pokémon ao mesmo tempo

APIS REAIS

`pokemon(nome)` devolve o JSON; `Promise.all` com pikachu e charmander; tipos com map.

Saída esperada: pikachu: electric (60) / charmander: fire (85)

16 Banco pelo código

DESAFIO

`banco(codigo)` na BrasilAPI tratando o 404 com `resposta.ok`.

Saída esperada: 001: Banco do Brasil S.A. / Banco não encontrado

> **Travou?** Volte ao exemplo da seção correspondente, rode o código no site e mude um valor por vez para entender o que cada parte faz.